

PROYECTO CLOUD & IA

NexaRetail Intelligence Platform

Plataforma de Inteligencia Artificial aplicada a empresa retail ficticia

Sprint 1

Sprint 2

AWS

OCI Free

INFORMACIÓN DEL PROYECTO	
Versión	1.0 — Documento Inicial
Fecha	Junio 2025
Estado	En planificación — Sprint 1 activo
Tecnologías Core	AWS (S3, Comprehend, Bedrock, Lambda, DynamoDB, Glue, Athena, QuickSight, Lex, Cognito)
Restricción Financiera	100% Free Tier / Capa gratuita de AWS
IaC	Terraform (para toda la infraestructura)
Gestión de Proyecto	Jira (tareas y sprints)
Comunicación	Discord (canal del equipo)

1. Resumen Ejecutivo

NexaRetail Intelligence Platform es un proyecto de portafolio cloud multi-sprint diseñado para demostrar competencias reales en AWS, inteligencia artificial aplicada y arquitectura cloud moderna. El proyecto simula una empresa retail mundial ficticia (estilo Walmart / Office Depot) llamada NexaRetail, con datos inventados de productos, empleados, franquicias, sucursales y clientes.

La idea central es construir, de forma incremental y documentada, un ecosistema de servicios cloud que una empresa real podría implementar: desde el análisis automatizado de reseñas de clientes (Sprint 1), hasta chatbots RAG inteligentes para clientes y empleados (Sprint 2), dashboards de BI, pipelines ETL, APIs REST seguras y más (Sprints futuros).

Todo el proyecto opera dentro de la capa gratuita de AWS y OCI, con infraestructura versionada en Terraform, gestión en Jira y comunicación en Discord. Cada sprint produce documentación, diagramas de arquitectura y código funcional.

2. La Empresa Ficticia: NexaRetail

Para que todos los sprints tengan datos reales y coherentes con los que trabajar, se crea una empresa ficticia llamada NexaRetail con la siguiente identidad:

IDENTIDAD DE LA EMPRESA
Nombre: NexaRetail S.A. de C.V. Tipo: Empresa retail multinacional (tiendas físicas + e-commerce) Giro: Venta de electrónica, artículos de oficina, hogar y tecnología Operación: Múltiples sucursales/franquicias en México y LATAM Datos generados: Inventados y coherentes para todos los sprints

2.1 Datos e Información que se Generará

La base de datos de NexaRetail se construye en el Sprint 1 y alimenta todos los sprints posteriores. A continuación se describe qué tipo de información se crea:

Datos de Negocio	Datos de Clientes
- Catálogo de productos con SKU, precio, categoría, stock - Lista de sucursales/franquicias con ubicación y gerente	- Base de datos de clientes registrados (nombre, historial de compras) - Reseñas y comentarios de productos en formato CSV

• Tabla de empleados con nombre, cargo, sucursal y antigüedad • Historial de ventas mensuales por producto y sucursal • Políticas internas, manuales de empleados (documentos TXT/PDF) • Normativas de la empresa (horarios, descuentos, devoluciones)	• Comentarios de sucursales (texto libre, positivo/negativo/neutro) • Preguntas frecuentes (FAQ) reales que hacen los clientes • Tickets de soporte con descripción y resolución • Evaluaciones de satisfacción (NPS simulado)
--	--

¿Por qué crear esta empresa ficticia? Tener datos propios y coherentes permite usar exactamente los mismos archivos en todos los sprints. En el Sprint 1 se analizan las reseñas CSV; en el Sprint 2 el chatbot RAG responde preguntas basándose en los documentos de la empresa; en sprints futuros, los mismos datos se transforman con ETL y se visualizan en dashboards. Es como construir sobre una base sólida en lugar de partir de cero cada vez.

3. Sprint 1 — Restaurant Intelligent Reviewer



DECISIÓN PENDIENTE DEL EQUIPO

El nombre del proyecto sigue siendo 'Restaurant Intelligent Reviewer', pero el equipo debe decidir si el contexto de datos será de un restaurante/franquicia de comida o de la empresa NexaRetail (retail). Se recomienda usar NexaRetail para que los datos del Sprint 1 sirvan directamente al Sprint 2 y sprints futuros sin necesidad de cambiar el dataset.

3.1 ¿Qué es este Sprint?

El Sprint 1 es el punto de entrada al proyecto. Su objetivo es construir un pipeline de análisis automático de texto (NLP) que procese reseñas o comentarios de clientes almacenados en formato CSV en Amazon S3, extraiga inteligencia del lenguaje natural con Amazon Comprehend y, de forma opcional, genere un resumen ejecutivo con recomendaciones usando Amazon Bedrock.

Este sprint es deliberadamente simple para que todo el equipo pueda completarlo, entender cómo funcionan los servicios de AWS de IA sin servidor, y construir la base de datos de la empresa ficticia que se utilizará en todos los sprints posteriores.

3.2 ¿Por qué empezar aquí?

- Es el proyecto más accesible: no requiere entrenar modelos de ML ni configurar infraestructura compleja.
- Permite aprender los servicios fundamentales de AWS (S3, Comprehend, Bedrock) de forma práctica.
- Genera los primeros datos reales del proyecto: los CSVs de reseñas y los resultados del análisis NLP.
- Al final del sprint, ya se tienen archivos, estructura de carpetas en S3 y conocimiento del flujo que el Sprint 2 va a ampliar con RAG y chatbots.
- Es 100% serverless: no hay servidores que apagar ni cargos inesperados.

3.3 ¿Qué Problema Resuelve?

Una empresa como NexaRetail recibe miles de comentarios de clientes cada mes. Leerlos manualmente es inviable. Este sprint automatiza ese proceso: toma un archivo CSV con cientos de reseñas y en minutos extrae:

- El sentimiento general de cada reseña (positivo, negativo, neutro o mixto).
- Las palabras clave más mencionadas (ej. 'tiempo de entrega', 'mal servicio', 'precio competitivo').
- Las entidades identificadas: nombres de productos, sucursales, personas o lugares mencionados.
- Un resumen ejecutivo opcional generado por IA con recomendaciones de negocio concretas.

3.4 Flujo del Sistema (Arquitectura)

FLUJO PASO A PASO	
1. PREPARACIÓN DE DATOS	
Se crea el CSV de reseñas de clientes NexaRetail con al menos 50-100 comentarios inventados, realistas y en español. Se sube el archivo a un bucket de Amazon S3.	
2. PROCESAMIENTO CON COMPREHEND	
Un script Python (boto3) envía las reseñas a Amazon Comprehend. Comprehend aplica tres análisis: Sentiment Analysis (¿es positivo?), Key Phrase Extraction (¿de qué habla?) y Entity Recognition (¿menciona algún producto o sucursal?).	
3. CONSOLIDACIÓN DE RESULTADOS	
Los resultados JSON de Comprehend se procesan y se consolidan en un nuevo CSV enriquecido con columnas de sentimiento, frases clave y entidades. Este archivo se guarda en S3.	
4. GENERACIÓN DE RESUMEN (OPCIONAL)	
Con los datos consolidados, se construye un prompt y se envía a Amazon Bedrock (modelo Nova Lite o Claude Haiku). Bedrock genera un resumen ejecutivo en lenguaje natural: '70% de reseñas negativas mencionan tiempo de entrega. Recomendación: revisar logística en sucursal Norte.'	

3.5 Servicios AWS Utilizados

SERVICIOS AWS — SPRINT 1	
Amazon S3	Almacenamiento de CSVs de entrada (reseñas) y salida (resultados enriquecidos). También guarda documentos de la empresa para usar en sprints futuros.
Amazon Comprehend	Motor NLP principal. Ejecuta análisis de sentimiento, extracción de frases clave y reconocimiento de entidades. No requiere entrenamiento.
Amazon Bedrock	(Opcional) Genera resúmenes ejecutivos y recomendaciones de negocio en lenguaje natural a partir de los resultados del análisis.
Python + boto3	Script de orquestación que conecta S3, Comprehend y Bedrock. Es el 'pegamento' del pipeline.

3.6 Conceptos Clave que el Equipo Aprenderá

Conceptos de IA / NLP	Habilidades Técnicas AWS
- NLP — Natural Language Processing - Sentiment Analysis (POSITIVE/NEGATIVE/NEUTRAL/MIXED) - Entity Recognition — detectar productos, lugares, personas - Key Phrase Extraction — ¿de qué habla la gente?	- Leer/escribir CSVs con Python - Subir y descargar archivos de S3 con boto3 - Invocar Amazon Comprehend desde Python - Interpretar respuestas JSON de Comprehend

- Batch vs Real-time processing
- Structured vs Unstructured data
- Prompt Engineering básico (para Bedrock)

- Construir prompts para modelos de lenguaje
- Invocar modelos de Bedrock con boto3
- Estructura de buckets y prefijos en S3

3.7 Entregables del Sprint 1

1. CSV de reseñas de clientes NexaRetail (datos inventados, coherentes y en español).
2. Script Python funcional que ejecuta el pipeline completo de análisis.
3. CSV de resultados enriquecidos guardado en S3.
4. Resumen ejecutivo generado por Bedrock (opcional pero recomendado).
5. Documentos base de NexaRetail en S3: catálogo, políticas, FAQs (para Sprint 2).
6. Diagrama de arquitectura del Sprint 1.
7. README actualizado con instrucciones para replicar el sprint.
8. Código Terraform para todos los recursos creados (S3, IAM roles, etc.).

4. Sprint 2 — Chatbot RAG con Amazon Bedrock

Una vez que el equipo domina el pipeline NLP del Sprint 1 y tiene los documentos de NexaRetail en S3, el Sprint 2 construye sobre esa base para crear un sistema de chatbot inteligente que puede responder preguntas en lenguaje natural usando la información real de la empresa.

La técnica central es RAG (Retrieval Augmented Generation): en lugar de que el modelo de IA 'memorice' información de NexaRetail, el sistema busca los documentos relevantes en tiempo real y los inyecta en el contexto del modelo para que pueda responder con información actual y precisa.

4.1 ¿Qué es RAG y por qué usarlo?

RAG — RETRIEVAL AUGMENTED GENERATION

El problema: Un modelo de IA como Claude o Titan no sabe nada de NexaRetail. Si le preguntas '¿cuál es el horario de la sucursal Centro?' simplemente no lo sabe.

La solución RAG: Cuando el usuario hace una pregunta, el sistema primero busca en los documentos de NexaRetail (guardados en S3 como vectores en Bedrock Knowledge Bases) los fragmentos más relevantes, los incluye en el mensaje al modelo, y el modelo responde usando esa información real. Es como darle al modelo el 'contexto correcto' antes de que responda.

Ventaja clave: No necesitas reentrenar el modelo cuando cambia la información. Solo actualizas los documentos en S3 y el sistema automáticamente responde con datos actualizados.

4.2 Dos Chatbots: Clientes vs. Corporativo

Se propone desarrollar dos chatbots especializados con acceso a diferentes bases de conocimiento:

 Chatbot de Clientes (NexaBot Customer)	 Chatbot Corporativo (NexaBot Interno)
• Responde preguntas sobre productos: precio, disponibilidad, especificaciones • Informa horarios de sucursales y ubicaciones • Resuelve dudas sobre políticas de devolución y garantías • Consulta el estado de pedidos (si se integra con la base de datos)	• Consultas de empleados sobre políticas internas de la empresa • Búsqueda en manuales de empleados y normativas • Análisis de comportamiento de clientes: '¿qué productos compran los clientes frecuentes?' • Consultas sobre métricas de ventas y rendimiento de sucursales

- | | |
|---|---|
| <ul style="list-style-type: none"> • FAQ general de la tienda en lenguaje amigable • Orientado al público general, lenguaje simple y amigable | <ul style="list-style-type: none"> • Soporte a gerentes: consejos basados en datos de ventas y tendencias • Acceso restringido solo a personal autorizado |
|---|---|

4.3 Arquitectura del Sprint 2

COMPONENTES TÉCNICOS

- Bedrock Knowledge Bases — Indexa los documentos de NexaRetail en S3 usando embeddings (Titan Embeddings V2). Convierte texto en vectores matemáticos para búsqueda semántica.
- S3 Vectors — Almacena los vectores generados (base de datos vectorial serverless, sin instancias que administrar).
- Lambda + Bedrock Converse API — Orquesta la conversación: recibe el mensaje del usuario, busca en Knowledge Bases, construye el prompt enriquecido y llama al modelo.
- Bedrock Guardrails — Filtros de seguridad: bloquea temas no permitidos (preguntas inapropiadas, intentos de jailbreak, información confidencial que no debe compartirse).
- Strands Agents SDK — Framework para crear agentes autónomos que no solo responden preguntas sino que también ejecutan acciones.
- Amazon Cognito — Autenticación para separar el acceso de clientes vs. empleados corporativos.

4.4 Agentes: Más Allá de Responder Preguntas

Los agentes de IA van un paso más allá del chatbot: pueden ejecutar acciones, no solo responder. Con el Strands Agents SDK de Bedrock, se propone implementar:

- Generación de documentos: el agente puede generar reportes de ventas en PDF o Excel basándose en los datos de la empresa.
- Agendado de reuniones: integración con calendarios para agendar citas con gerentes o reuniones de equipo.
- Predicciones de negocio: basándose en datos históricos de ventas, sugiere qué productos reabastecer o en qué sucursal se esperan más ventas.
- Alertas proactivas: si el agente detecta un patrón inusual en los datos, puede notificar por correo o mensaje.

4.5 Interfaz del Chatbot — Decisión Pendiente

OPCIONES PARA LA INTERFAZ DE USUARIO

Opción A — Amazon Lex: Servicio gestionado de AWS para interfaces conversacionales. Define 'intents' (intenciones del usuario) y 'slots' (datos a recopilar). Sin necesidad de entrenar modelos. Se integra con Lambda. Mejor para flujos estructurados (reservas, consultas específicas). Free tier: 10,000 requests de texto/mes.

Opción B — Interfaz web propia (HTML/React): Un chat básico en una página web estática servida desde S3 + CloudFront que llama directamente a una API Gateway conectada a Lambda + Bedrock. Más flexible y personalizable. Recomendada si se quiere control total sobre la interfaz.

Opción C — Open Source (Gradio / Streamlit): Bibliotecas Python que generan interfaces de chat listas en minutos. Gradio o Streamlit pueden ejecutarse localmente o desplegarse gratis. Ideal para demos rápidas y pruebas. No tan profesional para producción pero perfecto para el portafolio.

Recomendación del documento: *Iniciar con Opción B (web propia básica) + Opción C para demos rápidas locales, y considerar Lex para el Sprint posterior si se quiere demostrar esa tecnología específica.*

5. Sprints Futuros — Funcionalidades Adicionales

Una vez completados los Sprints 1 y 2, el proyecto puede expandirse de manera modular. Cada sprint futuro agrega una capa de valor independiente pero complementaria al sistema existente. A continuación se describen los sprints propuestos:

5.1 Data Lake Básico con S3 + Athena + Glue

SPRINT FUTURO — DATA LAKE	
Objetivo	Crear un repositorio centralizado donde todos los datos de NexaRetail (ventas, inventario, reseñas analizadas) puedan consultarse con SQL sin moverlos.
¿Qué hace?	AWS Glue Crawler escanea los archivos CSV/Parquet en S3 y crea automáticamente un catálogo de metadatos. Athena ejecuta consultas SQL directamente sobre esos archivos sin necesidad de cargar datos a una base de datos.
Valor en el proyecto	Centraliza todos los datos generados en los sprints anteriores. Cualquier análisis futuro parte de este Data Lake.
Valor en CV	Data Lake + Athena aparecen en vacantes de Data Engineer y Cloud Analyst.
Tecnologías	S3, AWS Glue Crawler, Athena, SQL, QuickSight (opcional)
Tiempo estimado	6-8 horas
Free Tier	Athena: primeros 5 TB de consultas gratis

5.2 Dashboard con Amazon QuickSight

SPRINT FUTURO — DASHBOARD BI	
Objetivo	Crear dashboards interactivos que visualicen los datos de NexaRetail: ventas por sucursal, sentimiento de clientes por mes, productos más mencionados en reseñas negativas.
¿Qué hace?	QuickSight se conecta directamente a los datos en S3/Athena. SPICE (motor en memoria) carga los datos para que las gráficas respondan al instante. Los dashboards son publicables con un link.
Valor en el proyecto	Convierte los datos crudos en visualizaciones que un gerente puede entender y usar para tomar decisiones.
Valor en CV	BI en cloud en roles de Analista de Datos. Complementa perfectamente el Data Lake.
Tecnologías	QuickSight, S3, Athena, SPICE, IAM
Tiempo estimado	6-8 horas
Free Tier	QuickSight: 1 autor gratis 30 días

5.3 Pipeline ETL Automático con Glue + Eventos S3

SPRINT FUTURO — ETL AUTOMATIZADO

Objetivo	Automatizar la transformación de datos: cuando se sube un nuevo CSV de ventas a S3, automáticamente se limpia, normaliza y guarda en formato Parquet optimizado.
¿Qué hace?	Un evento de S3 dispara una Lambda que lanza un Glue Job. El Job lee el CSV, aplica transformaciones Python (limpieza de nulos, normalización, joins) y guarda el resultado como Parquet. Parquet reduce el tamaño hasta 10x y acelera las consultas de Athena.
Valor en el proyecto	Automatiza el mantenimiento del Data Lake. Cada vez que llegan datos nuevos, se procesan solos.
Valor en CV	ETL automatizado es la tarea #1 de los Data Engineers.
Tecnologías	S3 Events, Lambda, Glue Jobs, Python, Parquet
Tiempo estimado	10-14 horas
Free Tier	Glue: 10 DPU-horas gratis/mes

5.4 API REST con Autenticación JWT en Lambda

SPRINT FUTURO — API REST SEGURA

Objetivo	Construir un backend completo con API REST donde los usuarios pueden registrarse, iniciar sesión y acceder a recursos protegidos de NexaRetail.
¿Qué hace?	Lambda ejecuta la lógica de negocio sin servidores. API Gateway expone los endpoints REST. Cognito maneja registro, login y reset de contraseña. DynamoDB almacena los datos. JWT (JSON Web Tokens) es el estándar de autenticación sin estado: el servidor genera un token firmado al hacer login y el cliente lo incluye en cada petición.
Valor en el proyecto	Es la capa de acceso seguro para el portal web de NexaRetail y para los chatbots. Quien usa el chatbot corporativo debe autenticarse.
Valor en CV	Auth + API + serverless es el stack más pedido en startups SaaS.
Tecnologías	Lambda, API Gateway, Cognito, DynamoDB, JWT
Tiempo estimado	12-16 horas
Free Tier	Cognito: 50,000 MAU gratis

5.5 Portal Web de NexaRetail

SPRINT FUTURO — PÁGINA WEB

Objetivo	Crear un sitio web estático que simule el portal público de NexaRetail: catálogo de productos, información de sucursales, y el chatbot de clientes integrado.
----------	---

¿Qué hace?	HTML/CSS/JS básico servido desde S3 + CloudFront con HTTPS. Se integra el chatbot de clientes del Sprint 2. La página muestra productos, sucursales y permite hacer preguntas al bot directamente desde la interfaz.
Valor en el proyecto	Es la cara visible del proyecto. Permite hacer demos impactantes en entrevistas.
Valor en CV	Demuestra que se puede construir una solución de principio a fin: datos, IA y interfaz.
Tecnologías	S3 Static Website, CloudFront, HTML/CSS/JS, API Gateway
Tiempo estimado	8-12 horas
Free Tier	S3 y CloudFront en free tier

5.6 Chatbot por Voz con Amazon Polly + Lex

SPRINT FUTURO — VOZ	
Objetivo	Agregar capacidad de voz al chatbot: el usuario habla y el sistema responde en audio.
¿Qué hace?	Amazon Lex maneja el reconocimiento de voz (speech-to-text). Amazon Polly convierte las respuestas del chatbot en audio (text-to-speech). Se integra en la interfaz web del portal.
Valor en el proyecto	Diferenciador importante. La mayoría de proyectos solo hacen texto; voz es un nivel más.
Valor en CV	Demuestra conocimiento de servicios de IA multimedia de AWS.
Tecnologías	Amazon Lex, Amazon Polly, Lambda, S3
Tiempo estimado	8-12 horas
Free Tier	Polly: 5M caracteres/mes gratis 12 meses; Lex: 10,000 solicitudes texto/mes gratis

5.7 Observabilidad: Métricas con Grafana / CloudWatch

SPRINT FUTURO — MONITOREO	
Objetivo	Monitorear el comportamiento de los modelos de IA y la infraestructura en tiempo real.
¿Qué hace?	CloudWatch recopila métricas de Bedrock (latencia, tokens consumidos, costo por petición). Las métricas se exportan a Prometheus y se visualizan en Grafana instalado en el servidor gratuito OCI Ampere A1. Se crean alertas: si la latencia supera 3 segundos o el costo diario supera \$5, notifica automáticamente.
Valor en el proyecto	Asegura que el sistema funciona correctamente en producción y permite detectar problemas antes de que afecten a los usuarios.
Valor en CV	MLOps + observabilidad es uno de los perfiles más escasos y mejor pagados.

Tecnologías	Bedrock, CloudWatch, Grafana, OCI Ampere A1 (gratis), Prometheus
Tiempo estimado	14-18 horas
Free Tier	Grafana en OCI A1 siempre gratis

6. Estándares del Proyecto

6.1 Restricción de Costos — Free Tier

POLÍTICA DE COSTO CERO

Todo el proyecto debe operar dentro de la capa gratuita de AWS y OCI. Antes de usar cualquier servicio nuevo, verificar su free tier en <https://aws.amazon.com/free>. Si un sprint requiere un servicio fuera del free tier, se documenta el costo estimado y se busca alternativa.

- Nunca dejar instancias EC2 corriendo sin necesidad (apagar después de usar).
- Usar S3 lifecycle policies para mover datos viejos a Glacier si se acumula mucho.
- Bedrock: usar modelos de bajo costo (Nova Lite, Haiku) para pruebas; solo usar modelos avanzados cuando sea necesario.
- Revisar el AWS Cost Explorer semanalmente para detectar cargos inesperados.
- OCI ofrece un servidor ARM gratuito para siempre (Ampere A1) — ideal para Grafana y servicios auxiliares.

6.2 Infraestructura como Código — Terraform

POR QUÉ TERRAFORM EN ESTE PROYECTO

Todos los recursos creados en AWS deben tener su equivalente en código Terraform. Esto no es opcional — es parte del aprendizaje y de la seguridad del proyecto.

¿Por qué Terraform y no solo la consola de AWS?

- Si alguien borra accidentalmente un recurso, se puede recrear en minutos con terraform apply.
- El equipo aprende IaC (Infrastructure as Code), una habilidad muy demandada en el mercado.
- El código Terraform documenta exactamente qué existe en la nube y cómo está configurado.
- Permite replicar toda la infraestructura en una cuenta nueva sin configuración manual.

Estructura de carpetas recomendada:

- terraform/sprint1/ — Recursos del Sprint 1 (S3, IAM)
- terraform/sprint2/ — Recursos del Sprint 2 (Bedrock KB, Lambda, Cognito)
- terraform/modules/ — Módulos reutilizables (ej. módulo S3 estándar)

6.3 Documentación Continua

- Cada sprint debe actualizar el README del repositorio con instrucciones para replicarlo.
- Cada sprint debe incluir un diagrama de arquitectura actualizado (draw.io o Lucidchart recomendado).
- Los cambios importantes se documentan en el propio documento de proyecto (este archivo).
- Todo el código debe tener comentarios explicativos en español para que cualquier miembro pueda entenderlo.

6.4 Herramientas de Gestión y Comunicación

Jira — Gestión del Proyecto	Discord — Comunicación del Equipo
• Un épico por sprint (ej. 'Sprint 1 — NLP Pipeline') • Tareas individuales asignadas a cada miembro del equipo • Estimación de tiempo en horas para cada tarea • Revisión semanal del estado del sprint (sprint review) • Backlog ordenado con los sprints futuros	• Canal #general para comunicación del día a día • Canal #ayuda-técnica para resolver dudas de código • Canal #arquitectura para discutir decisiones de diseño • Canal #sprints para actualizaciones de avance • Llamadas de revisión semanales (voz en Discord)

7. Roadmap del Proyecto

Sprint	Nombre	Tecnologías	Dificultad / Tiempo
Sprint 1	Restaurant Intelligent Reviewer (NLP + Comprehend)	S3, Comprehend, Bedrock, Python	Baja — 10-15h
Sprint 2	Chatbot RAG con Bedrock (Clientes + Corporativo)	Bedrock KB, S3 Vectors, Lambda, Cognito, Strands SDK	Alta — 20-30h
F-1	Data Lake: S3 + Athena + Glue	S3, Glue Crawler, Athena, SQL	Principiante — 6-8h
F-2	Dashboard BI con QuickSight	QuickSight, S3, Athena, SPICE	Principiante — 6-8h
F-3	ETL Automático Glue + S3 Events	S3 Events, Lambda, Glue Jobs, Parquet	Intermedio — 10-14h
F-4	API REST con JWT + Cognito	Lambda, API Gateway, Cognito, DynamoDB	Intermedio — 12-16h
F-5	Portal Web NexaRetail	S3 Static, CloudFront, HTML/JS, API GW	Intermedio — 8-12h
F-6	Chatbot por Voz (Polly + Lex)	Lex, Polly, Lambda	Intermedio — 8-12h
F-7	Observabilidad con Grafana	CloudWatch, Prometheus, Grafana, OCI	Avanzado — 14-18h

8. Decisiones Pendientes del Equipo

Las siguientes decisiones deben ser tomadas por el equipo antes o durante el Sprint 1:

DECISIONES A TOMAR	
Decisión 1	¿El Sprint 1 usa datos de restaurante/franquicia de comida o de NexaRetail (retail)? Recomendación: usar NexaRetail para que los datos sirvan en todos los sprints.
Decisión 2	¿Cuántos comentarios/reseñas de clientes se generan para el CSV inicial? Sugerencia: mínimo 100, idealmente 200-300 para resultados más interesantes.
Decisión 3	¿Se implementa Amazon Bedrock en el Sprint 1 o se deja como opcional? El costo es mínimo en free tier pero es un paso adicional de configuración.
Decisión 4	¿Qué interfaz se usa para el chatbot del Sprint 2: Amazon Lex, página web propia, o Streamlit/Gradio? Esta decisión define cuánto esfuerzo va en frontend.
Decisión 5	¿Se hacen los dos chatbots (cliente + corporativo) en el Sprint 2 o se hace uno primero y el otro como sprint adicional?
Decisión 6	¿Cuál es el nombre final de la empresa ficticia? Se propone NexaRetail pero el equipo puede elegir.

9. Notas Finales

Este documento es un punto de partida, no una especificación cerrada. El proyecto es ágil: el equipo puede ajustar el alcance de cada sprint según el tiempo disponible y los aprendizajes obtenidos.

Lo más importante es mantener el ciclo: construir → documentar → aprender → mejorar. Cada sprint que se complete, bien documentado y con código funcional, es un activo real en el portafolio de cada miembro del equipo.

El objetivo final no es solo construir una plataforma técnica, sino poder explicar en una entrevista de trabajo exactamente qué construiste, por qué lo construiste así, qué problemas resolviste y qué aprendiste en el proceso. Este documento, junto con el código y los diagramas, es la evidencia de ese trabajo.